Presentation and demonstration guide

Opening pitch

Our project asks what happens when a database administrator changes a record and rewrites its audit entry. Both database objects may agree, so checking one against the other is not enough. We keep a separate witness receipt for each accepted transition. The verifier compares that receipt with the current row and audit event before checking its Merkle checkpoint. A legitimate update waiting for a checkpoint remains pending. A change that disagrees with the witness produces a specific inconsistency. The system then refuses to anchor that disagreement. We tested these behaviours in a local SQLite laboratory. The prototype passed 32 tests, including row-and-audit rewrites, rollback and deletion. Our proposed research contribution is the reconciliation workflow. We recognise that database receipts and asynchronous Merkle anchoring already have close prior art, so the patent direction still requires a formal search.

Demonstration sequence

Start a fresh data directory: python -m demo.server --port 18877 --data data/presentation-1. Open http://127.0.0.1:18877. Create ID 1 with amount 250.00 and ID 2 with amount 100.00. Show the pending verdict. Click Anchor pending receipts and then Inspect. Point to the leaf, sibling path and root. Update ID 1 to 300.00 through Save with receipt. Show that a legitimate update returns to pending, then anchor again. Select Rewrite row and database audit, inject the fault on ID 1 and inspect it. The result should be AUDIT_TAMPERED. Try anchoring again and show ANCHOR_BLOCKED. Use a fresh data directory for each rehearsal, or different directories for separate fault scenarios. Do not delete or alter the original PostgreSQL data.

Answers to likely questions

What is actually new? The candidate is the explicit reconciliation rule and the decision to block a checkpoint when live data, audit evidence and witness history disagree. Merkle trees and external roots are established. We have not proven that the candidate is absent from prior patents.

Why use a Merkle tree if the witness already stores receipts? Receipts preserve the transition evidence. A checkpoint gives a compact batch commitment and an inclusion proof. In this prototype both reside in the witness file, so the security still depends on trusting that store. A production deployment needs separately administered storage or an external anchor.

Does this stop an administrator from editing data? It detects the demonstrated database-only edits. It does not stop SQL commands. Full host control, witness compromise or malicious writes accepted through the application are outside the claimed protection.

Is verification O(log n)? The Merkle path has logarithmic depth in the batch size. The full implementation scans histories and checkpoint documents, so its total cost is larger. The measured 300-record checkpoint time makes this limitation visible.

What happens if the program crashes? A receipt committed before a failed business transaction remains visible as missing evidence. Automatic repair and distributed atomic commit are not implemented. Manual investigation is required.

Is the system asynchronous? The original repository contains an asynchronous PostgreSQL worker. The tested receipt lab separates writes from manually requested checkpoints. The witness receipt itself is synchronous. Do not claim that the new workflow has zero write overhead.

Can we claim a patent outcome? Only after the required institutional review and patent search support a defensible claim. This report uses the implementation category because first-review patent approval has not been established in the available material.